Running the server as a container

The published image is `registry.badassium.com/fatalexception/yarg-song-server`, tagged `latest` and the commit short sha. It is a distroless static binary — **8.6 MB**, no shell, running as `nonroot`.

This document records the first real deployment, on vault2, 2026-09-05.

What this deployment proved, that nothing had proved before

The multi-arch CI job verifies the image config's architecture and the ELF machine type of the binary inside. Both are checks on *bytes*. Until this deployment **the published image had never actually been run** — not by CI, not by anyone. It now has:

```
library indexed songs=22 distinct_charts=22 duplicate_packages=0 problems=0 took=7ms
listening addr=:8080 songs=/songs data=/data version=827e0fe
```

`version=827e0fe` is how you know the image is the commit it claims to be.

It also moved the server off the development machine for the first time. Every earlier measurement was `127.0.0.1`, which cannot fail in any of the ways a real network can.

The deployment

```
# on the host
mkdir -p /mnt/cache/appdata/yarg-song-server/songs \
         /mnt/cache/appdata/yarg-song-server/data
chown -R 65532:65532 /mnt/cache/appdata/yarg-song-server/data

docker run -d --name yarg-song-server \
  --restart unless-stopped \
  -p 8099:8080 \
  -v /mnt/cache/appdata/yarg-song-server/songs:/songs:ro \
  -v /mnt/cache/appdata/yarg-song-server/data:/data \
  registry.badassium.com/fatalexception/yarg-song-server:latest
```

Three things there are not decoration:

- **`chown 65532`.** The image is distroless and runs as `nonroot`, uid 65532. `/data` holds the on-demand pack cache and must be writable by that uid or the server starts fine and then fails the first time somebody asks for a song that is stored as a loose folder. `/songs` stays root-owned and is mounted `:ro`, because the server never writes to the library and should not be able to.
- **`/mnt/cache/...`, not `/mnt/user/...`.** An absolute pool path, not the shfs union. This host has a documented history of SQLite-on-FUSE corruption, and `/mnt/user` was **99% full** at deployment (568 GB of 29 TB) while the cache pool had 1.4 TB free.
- **`-p 8099:8080`.** 8099 was free; the container listens on 8080 internally. Checked against `ss -ltn` rather than assumed — this host had 59 containers running and 76 ports already in LISTEN.

Registry authentication

The host's stored credential was stale and `docker pull` failed with `unauthorized: HTTP Basic: Access denied`. A GitLab personal access token with the **`api`** scope works for a registry pull — measured; `read_registry` is the documented scope but was not required here.

Log in without putting the secret in `argv`, where `ps` would show it:

```
docker login registry.badassium.com -u <user> --password-stdin <<'ENDTOK'
<token>
ENDTOK
```

The token comes from 1Password at run time and is never written to a file, a commit, or this document.

End-to-end result, 2026-09-05

Step	Result
Pull and start on vault2	running, 0 restarts, indexed 22 songs in 7 ms
`GET /healthz` from the host	200
`yarg-sync` from ENG-1 over Tailscale	22 downloaded, 0 failed, 229,515 bytes in **341 ms**
Byte count vs the localhost run	**identical** — same archives, different machine
Second sync	0 downloaded, 22 already present, **0 bytes**, 19 ms
Unmodified YARG on the synced folder	**20 accepted, 2 refused**

The two refusals were the same two, for the same reasons, as every earlier run — `no notes found` and `no audio accompanying the chart file`, both independently flagged by our own scanner.

The client reached the server by its **Tailscale name**, not a LAN literal, so the same command works from anywhere ENG-1 happens to be. A `192.168.2.x` address would have worked at the time of the test and failed from the office.

The pack cache bound, verified here rather than in a test

`pack_cache_max` shipped with six red-proofed tests and did not work. Every test inserted, so every test reached eviction through the insert path; this deployment's cache was already full, every request was a hit, nothing was ever packed, and the bound was never enforced. Deploying it found that in under a minute.

After the fix (`6bcf4ec`, enforce at start as well as on insert), running deliberately tight at **102,400 bytes** against a library that packs to 229,515:

Moment	Cache
Before restart, from the broken build	229,515 bytes, 22 archives
Immediately after start, having served nothing	**67,681 bytes, 6 archives**
After serving all 22 songs	67,681 bytes, 6 archives
Songs the client received	**22 of 22, 0 failed**

Three things in that table, in order of importance:

- The cache dropped from 22 archives to 6 **before a single request arrived**. That is the start-up enforcement, and it is the case the tests all missed.

- It stayed at 6 while serving 22 songs, so eviction kept pace with insertion rather than merely happening once.
- **Every song was still delivered and verified.** Sixteen of the twenty-two had to be re-packed mid-sync because eviction had removed them, and the client — which re-derives each song's identity from the bytes it receives — accepted all of them. That is the concrete evidence that eviction costs a re-pack and never data.

What this still does not prove

- **arm64 has never been executed.** The image is multi-arch and the arm64 binary is verified by ELF machine type, but no ARM hardware has run it. The Pi is the missing piece: `plzpi` was unreachable and there is no Pi on the tailnet at all. Until then, "portable to Raspberry Pi" is a claim.
- **The Phase 2b exit criterion needs two clients.** Only ENG-1 has YARG installed; r7 has no YARG, no settings directory and no Go. So "two machines running stock YARG both see the same library" remains unmeasured — this run did the server half and one client.
- One library, 22 songs, 224 KB. Nothing here says anything about a real collection's size or scan time.